Quality Assured Vision

Matty Luriz (23206450)
Mohammed Yaacoub Abou Chlih (21138726)

What is Quality Assured Vision's mission? And for who?

Embedded Computer Vision systems, used in robots, drones, and smart cameras use resource strained hardware with intensive vision models to make real time decisions. The big gap: a detection is accurate but late, a system that degrades under load is a failure in real-time context. Some stakeholders include Engineers, QA Engineers, End users, and Safety reviewers. The scope was to look at key software quality issues, those being the lack of defined, testable NFRs, lack of methods to measure latency, framerate, and resource usage, the difficulty differentiating between "accurate model" from "the system meets real time requirements", performance degradation under load, and the absence of defect management processes.

Key functional and non functional requirements

- **Functional Requirements**

FR-02: Detect and classify objects in each frame (Passed and prints the scores in FR-07)

FR-06: Configurable simulated resource constraint before a run (Passed but has no memory cap)

FR-07: Summary report of accuracy and performance metrics (Passed but has no accuracy metrics)

- **Non-functional Requirements**

NFR-02: Atleast 95% of frames sustain 15 FPS (Fail, 0% of frames sustain 15 FPS)

NFR-06: Results within 2% variance across 5 repeated runs (Pass, 0% variance)

NFR-08: All defined error conditions handled without crashing (Pass)

Final prototype

GUI screenshot, implemented features

Assessment 1 vs Assessment 2

NFR-03 measurement, four dedicated error-handling tests, NFR-09 threshold revision, NFR-02/04/06/07 coverage

Testing approach and coverage

Test levels, 16 test cases, how requirements trace to tests

Test results

14 passed, 2 failed, with the Status convention in one line; the profile/load matrix as a table or bar chart

Automation and CI

Green Actions run screenshot, quality gates, why MAUI is excluded

Quality Testing

Usability and performance findings, including the two usability observations

Defects and root cause analysis

The concurrency bug as the lead RCA, plus the others in the register

Metrics and release decision

Pass rate, requirement coverage, defect distribution, release verdict

Copilot and AI

What each tool did, where it was wrong, how you checked, risks

Lessons and future work

Whether 15 FPS is realistic, descriptive labels, repeated runs, user-supplied datasets, real hardware

Contributions and questions

Who did what, with commit evidence

Short video of baseline run + constrained/highload run